МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №1

з курсу: „Комп'ютерні мережі”

виконав студент III курсу
групи ДА-11
Сидько Нікіта
Варіант 24

КИЇВ - 2024

Мета роботи:

- Ознайомлення з роботою мережевих транспортних протоколів TCP і UDP;
- Ознайомлення з сокетами і елементами API сокетів;
- Написання програмних модулів клієнт і сервер для організації обміну даними за протоколами TCP і UDP.

Завдання

1)Ознайомтесь з теоретичними відомостями**.**

2)Вибрати варіант завдання за номером списку груп. Завдання полягає у тому, щоб переслати N чисел типу Тип1 і M чисел типу Тип2.

3)Написати програму для передачі даних між сервером та клієнтом TCP

4)Написати програму для пересилання даних між сервером і клієнтом UDP.

5)Перевірити її роботу, запам’ятати скріншоти результатів.

Теоретичні відомості

TCP (Transmission Control Protocol) і UDP (User Datagram Protocol) - це два основних протоколи транспортного рівня в мережевій моделі OSI (Open Systems Interconnection).

TCP

- TCP є з'єднанням та орієнтованим на потік протоколом. Він забезпечує надійну, послідовну та орієнтовану на потік передачу даних між двома пристроями.
- TCP використовує механізми як встановлення з'єднання, контролю потоку, підтвердження отриманих пакетів та перевірки цілісності даних.

Це робить TCP більш надійним протоколом, але при цьому менш ефективним у використанні ресурсів мережі порівняно з UDP.
UDP:

UDP є простим протоколом без з'єднання та без забезпечення надійної доставки даних.

- Він передає незалежні пакети даних без будь-якого контролю потоку, підтвердження доставки або відновлення порядку.
- UDP швидше за TCP та менше витрачає ресурси мережі, але дозволяє втрату даних або їх дублювання.

Сокети - це механізм в програмуванні для обміну даними між процесами через мережу. Вони використовуються для створення з'єднань між клієнтами і серверами та передачі даних між ними.

Елементи API сокетів - це набір програмних інтерфейсів, доступних розробникам для роботи з сокетами.
Ці інтерфейси включають у себе функції для створення, налаштування, з'єднання, передачі та отримання даних через сокети.
Мови програмування надають свої власні API для роботи з сокетами, наприклад, Winsock для C++ на платформі Windows

Написання програмних модулів на C++ клієнт і сервер для організації обміну даними за протоколами TCP і UDP:
Для написання програмних модулів на C++ для клієнта і сервера для організації обміну даними за протоколами TCP і UDP, вам зазвичай потрібно:

Використати відповідні бібліотеки або API для мови C++, щоб створити сокети і встановити з'єднання з іншими пристроями через TCP або UDP.

Налаштувати сокети для передачі та отримання даних.Написати код для обробки даних, які передаються між клієнтом і сервером через вибраний протокол.

У мові програмування C/C++, для організації сокетів, зазвичай використовуються функції з бібліотеки **socket.h**. Ось деякі з найбільш важливих функцій для цієї мети:

1. **Socket ()**: Ця функція використовується для створення сокету і повертає файловий дескриптор, який ідентифікує сокет в системі.

2. **Bind ()**: Ця функція прив'язує сокет до певної адреси.

3. **Listen ()**: Ця функція використовується на стороні сервера для переключення сокету у режим очікування на з'єднання.

4. **Accept ()**: Ця функція приймає вхідний з'єднання на стороні сервера.

5. **Connect ()**: Ця функція встановлює з'єднання з сервером на стороні клієнта.

6. **Send () / sendto()**: Ці функції використовуються для відправки даних через сокет.

7. **Recv () / recvfrom()**: Ці функції використовуються для отримання даних через сокет.

8. **Close ()**: Ця функція закриває сокет.

Варіант

№ варіанта	N	Тип1	M	Тип
24.	10	int	5	int

Переслати N чисел типу Тип1 і M чисел типу Тип2.

Передача даних між сервером та клієнтом TPC

TCP – клієнт

```
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib")

int main() {
    WSADATA wsaData;
    int iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);
    if (iResult != 0) {
        std::cerr << "WSAStartup failed: " << iResult <<
std::endl;
        return 1;
    }

    SOCKET ConnectSocket = socket(AF_INET, SOCK_STREAM,
IPPROTO_TCP);
    if (ConnectSocket == INVALID_SOCKET) {
        std::cerr << "Error at socket(): " << WSAGetLastError()
<< std::endl;
        WSACleanup();
        return 1;
    }

    sockaddr_in clientService;
    clientService.sin_family = AF_INET;
    clientService.sin_port = htons(27015);
```

```
    inet_pton(AF_INET, "192.168.0.106",
&clientService.sin_addr);

    iResult = connect(ConnectSocket, (SOCKADDR*)&clientService,
sizeof(clientService));
    if (iResult == SOCKET_ERROR) {
        std::cerr << "Connect failed: " << WSAGetLastError() <<
std::endl;
        closesocket(ConnectSocket);
        WSACleanup();
        return 1;
    }

    int numbers[15];
    iResult = recv(ConnectSocket, (char*)numbers,
sizeof(numbers), 0);
    if (iResult > 0) {
        std::cout << "Received numbers:" << std::endl;
        for (int i = 0; i < 15; i++) {
            std::cout << "Number " << i + 1 << ": " <<
numbers[i] << std::endl;
        }
    } else if (iResult == 0) {
        std::cout << "Connection closed" << std::endl;
    } else {
        std::cerr << "recv failed: " << WSAGetLastError() <<
std::endl;
    }

    closesocket(ConnectSocket);
    WSACleanup();
    return 0;
}
```

TCP – сервер

```
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib")

int main() {
    WSADATA wsaData;
    int iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);
    if (iResult != 0) {
        std::cerr << "WSAStartup failed: " << iResult <<
std::endl;
        return 1;
    }

    SOCKET ListenSocket = socket(AF_INET, SOCK_STREAM,
IPPROTO_TCP);
```

```
    if (ListenSocket == INVALID_SOCKET) {
        std::cerr << "Error at socket(): " << WSAGetLastError()
<< std::endl;
        WSACleanup();
        return 1;
    }

    sockaddr_in service;
    service.sin_family = AF_INET;
    service.sin_addr.s_addr = INADDR_ANY;
    service.sin_port = htons(27015);

    iResult = bind(ListenSocket, (SOCKADDR *)&service,
sizeof(service));
    if (iResult == SOCKET_ERROR) {
        std::cerr << "Bind failed with error: " <<
WSAGetLastError() << std::endl;
        closesocket(ListenSocket);
        WSACleanup();
        return 1;
    }

    iResult = listen(ListenSocket, SOMAXCONN);
    if (iResult == SOCKET_ERROR) {
        std::cerr << "Listen failed with error: " <<
WSAGetLastError() << std::endl;
        closesocket(ListenSocket);
        WSACleanup();
        return 1;
    }

    std::cout << "Waiting for client connections..." <<
std::endl;

    SOCKET ClientSocket = accept(ListenSocket, NULL, NULL);
    if (ClientSocket == INVALID_SOCKET) {
        std::cerr << "Accept failed: " << WSAGetLastError() <<
std::endl;
        closesocket(ListenSocket);
        WSACleanup();
        return 1;
    }

    std::cout << "Data transmission to client: Starting" <<
std::endl;
    int numbers[15] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,
13, 14, 15};
    send(ClientSocket, (char*)numbers, sizeof(numbers), 0);
    std::cout << "Data transmission to client: Completed" <<
std::endl;

    closesocket(ClientSocket);
    closesocket(ListenSocket);
```

```
    WSACleanup();
    return 0;
}
```

Передача даних між сервером та клієнтом UDP

UDP – клієнт

```
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib")

int main() {
    WSADATA wsData;
    int iResult = WSAStartup(MAKEWORD(2, 2), &wsData);
    if (iResult != 0) {
        std::cerr << "WSAStartup failed: " << iResult <<
std::endl;
        return 1;
    }

    SOCKET udpclsocket = socket(AF_INET, SOCK_DGRAM,
IPPROTO_UDP);
    if (udpclsocket == INVALID_SOCKET) {
        std::cerr << "Error at socket(): " << WSAGetLastError()
<< std::endl;
        WSACleanup();
        return 1;
    }

    sockaddr_in linkto;
    linkto.sin_family = AF_INET;
    linkto.sin_port = htons(223);
    inet_pton(AF_INET, "192.168.0.106", &linkto.sin_addr);
```

```
    int numbers[15] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,
13, 14, 15};
    std::cout << "Sending data to the server: Starting" <<
std::endl;
    sendto(udpclsocket, (char*)numbers, sizeof(numbers), 0,
(sockaddr*)&linkto, sizeof(linkto));
    std::cout << "Sending data to the server: Completed" <<
std::endl;

    closesocket(udpclsocket);
    WSACleanup();
    return 0;
}
```

UDP – сервер

```
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib")

int main() {
    WSADATA wsData;
    int iResult = WSAStartup(MAKEWORD(2, 2), &wsData);
    if (iResult != 0) {
        std::cerr << "WSAStartup failed: " << iResult <<
std::endl;
        return 1;
    }

    SOCKET udpservsocket = socket(AF_INET, SOCK_DGRAM,
IPPROTO_UDP);
    if (udpservsocket == INVALID_SOCKET) {
        std::cerr << "Error at socket(): " << WSAGetLastError()
<< std::endl;
        WSACleanup();
        return 1;
    }
    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(223);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    iResult = bind(udpservsocket, (sockaddr*)&serverAddr,
sizeof(serverAddr));
    if (iResult == SOCKET_ERROR) {
```

```
        std::cerr << "Bind failed with error: " <<
WSAGetLastError() << std::endl;
        closesocket(udpservsocket);
        WSACleanup();
        return 1;
    }
    sockaddr_in client;
    int clientSize = sizeof(client);
    int numbers[15];
    iResult = recvfrom(udpservsocket, (char*)numbers,
sizeof(numbers), 0, (sockaddr*)&client, &clientSize);
    if (iResult == SOCKET_ERROR) {
        std::cerr << "recvfrom failed with error: " <<
WSAGetLastError() << std::endl;
        closesocket(udpservsocket);
        WSACleanup();
        return 1;
    }
    std::cout << "Received numbers:" << std::endl;
    for (int i = 0; i < 15; ++i) {
        std::cout << "Number " << i + 1 << ": " << numbers[i] <<
std::endl;
    }
    closesocket(udpservsocket);
    WSACleanup();
    return 0;
}
```

Перевірити її роботу, запам’ятати скріншоти результатів.

TCP

Ініціалізація Winsock у сервері та клієнті:
Запускаємо функцію WSAStartup з версією 2.2 для ініціалізації використання Winsock DLL. Це необхідно для подальшої роботи з мережевими сокетами у Windows.
У разі помилки при ініціалізації (якщо WSAStartup повертає код, відмінний від 0), виводиться повідомлення про помилку, і програма завершується.

Сервер

Створення сокета
Створюється TCP сокет за допомогою функції socket(), використовуючи протокол IPPROTO_TCP. Це вказує на використання протоколу TCP. У випадку помилки, якщо socket() повертає INVALID_SOCKET, виводиться повідомлення про помилку, і програма завершується.

Прив'язка сокета до адреси:

Сокет прив'язується до локальної адреси та порту (27015), використовуючи функцію bind(). Це дозволяє серверу приймати з'єднання на цьому порту від клієнтів.

Слухання вхідних з'єднань:
Функція listen() використовується для встановлення сокета в режим прослуховування вхідних з'єднань. Сокет переходить в стан, де він може приймати вхідні з'єднання.

Прийняття вхідного з'єднання:
Функція accept() використовується для прийняття вхідного з'єднання від клієнта. Ця функція блокує виконання до моменту встановлення з'єднання.

Відправлення даних:
Після встановлення з'єднання сервер відправляє дані клієнту через функцію send().

Клієнт

Створення сокета:
Аналогічно серверу, створюється TCP сокет для з'єднання з сервером.

З'єднання з сервером:
Використовуючи функцію connect(), клієнт встановлює з'єднання з сервером, адреса та порт якого вказані у структурі sockaddr_in.

Отримання даних:
Функція recv() використовується для отримання даних від сервера. Дані читаються у масив, і потім виводяться у консоль.
Закриття сокетів та очищення ресурсів:

Обидва сервер та клієнт закривають свої сокети за допомогою closesocket() і виконують очищення ресурсів Winsock за допомогою WSACleanup() після завершення передачі даних.Таким чином, сервер і клієнт працюють разом для встановлення з'єднання та обміну даними в середовищі TCP. Сервер чекає на з'єднання від клієнта, а клієнт ініціює з'єднання та отримує дані від сервера.
TCP – клієнт

```
Received numbers:
Number 1: 1
Number 2: 2
Number 3: 3
Number 4: 4
Number 5: 5
Number 6: 6
Number 7: 7
Number 8: 8
Number 9: 9
Number 10: 10
Number 11: 11
Number 12: 12
Number 13: 13
Number 14: 14
Number 15: 15
```

TCP – сервер

```
Waiting for client connections...
Data transmission to client: Starting
Data transmission to client: Completed

Process finished with exit code 0
```

UDP

Ініціалізація Winsock у сервері та клієнті:
Початкова ініціалізація Winsock DLL через WSAStartup з версією 2.2, необхідна для роботи з мережевими сокетами у Windows.
У разі помилки (ненульовий результат WSAStartup), виводиться повідомлення про помилку, і програма завершується.

Сервер

Створення сокета:
Створюється UDP сокет за допомогою socket(), з використанням SOCK_DGRAM та IPPROTO_UDP, що позначає використання протоколу UDP. У разі помилки (якщо socket() повертає INVALID_SOCKET), виводиться повідомлення про помилку, і програма завершується.
Прив'язка сокета до адреси:
Сокет прив'язується до порту 223 на будь-якому доступному мережевому інтерфейсі, використовуючи функцію bind(). У разі помилки (bind() повертає SOCKET_ERROR), виводиться повідомлення про помилку, і програма завершується.
Прийняття даних:
Сервер використовує recvfrom() для прийняття даних. Ця функція дозволяє отримати дані та адресу відправника. У разі помилки, виводиться повідомлення про помилку, і програма завершується.
Виведення отриманих даних:
Отримані дані виводяться у консоль.

Закриття сокета:

Після завершення роботи сокет закривається, і відбувається очищення Winsock ресурсів через WSACleanup().

Клієнт

Створення сокета:
Аналогічно серверу, створюється UDP сокет.
Налаштування адреси сервера:
Адреса та порт сервера налаштовуються. ІП адреса сервера встановлюється як "192.168.0.106".
Відправлення даних:
Клієнт відправляє масив чисел до сервера використовуючи sendto().
У разі помилки, виводиться повідомлення про помилку, і програма завершується.
Закриття сокета:
Сокет закривається, і відбувається очищення Winsock ресурсів через WSACleanup().
Взаємодія між сервером та клієнтом:
Сервер чекає на дані від клієнта. Коли дані надходять, сервер приймає їх та виводить інформацію про отримані числа. Клієнт ініціює відправлення даних до сервера і отримує підтвердження про успішну передачу. Обидва компоненти використовують безз'єднанний протокол UDP, що забезпечує швидку відправку без підтвердження доставки, але без гарантії надійності або порядку отримання пакетів.

UDP – клієнт

```
Sending data to the server: Starting
Sending data to the server: Completed

Process finished with exit code 0
```

UDP – сервер

```
Received numbers:
Number 1: 1
Number 2: 2
Number 3: 3
Number 4: 4
Number 5: 5
Number 6: 6
Number 7: 7
Number 8: 8
Number 9: 9
Number 10: 10
Number 11: 11
Number 12: 12
Number 13: 13
Number 14: 14
Number 15: 15

Process finished with exit code 0
```

Висновок

У ході виконання першої лабораторної роботи було вивчено принципи роботи сокетів і ключові аспекти протоколів TCP та UDP. Ми дізналися що:

- TCP є надійним протоколом з'єднання, який забезпечує доставку даних без помилок та втрат, завдяки використанню механізмів контролю потоку та підтвердження пакетів. TCP вимагає попереднього встановлення з'єднання між клієнтом та сервером і ідеально підходить для задач, де критично важливі надійність і послідовність передачі даних, наприклад, для передачі файлів або роботи веб-сайтів
- UDP, будучи протоколом без з'єднання, не гарантує надійності та порядку в доставці даних, що робить його швидшим та менш ресурсозатратним, оскільки він не використовує механізми контролю потоку чи підтвердження пакетів

Під час виконання завдання було розроблено два коди для передачі даних типів int між сервером та клієнтом за допомогою TCP та UDP протоколів.